Anexo de Trazabilidad del Copiloto de IA

Registro auditable de bloqueos técnicos, contraste crítico y resolución

Proyecto Final — Sistema de Triaje Clínico Preliminar

Artificial Intelligence Essentials (AIE) — Summer Camp 2026

Escuela Politécnica Nacional Célula de Ciberseguridad CyberMinds

Autor: [NOMBRE COMPLETO]

Fecha: 25 de agosto de 2026

1. Alcance de este documento

Este anexo documenta el uso de asistentes de inteligencia artificial como copiloto durante el desarrollo. **No es una transcripción de conversación.** Recoge únicamente los bloqueos técnicos que exigieron decisión de ingeniería, redactados en registro profesional, y consigna de forma explícita qué propuso el modelo, qué se aceptó, qué se rechazó y por qué.

El criterio de inclusión fue la sustancia técnica: se documentan ocho bloqueos que alteraron el diseño, no la totalidad de las consultas realizadas. Los incidentes menores —errores de sintaxis, ajustes de formato, consultas de referencia— se omiten por carecer de valor de trazabilidad.

2. Herramientas y modelos empleados

Herramienta	Modelo	Rol en el proyecto
Claude Code (CLI)	Claude Sonnet 5 · Claude Opus 5	Copiloto de desarrollo: arquitectura, codificación, depuración
Asistente conversacional	—	Ideación inicial del concepto, previa al desarrollo
Groq Cloud	`openai/gpt-oss-120b`	**Componente del producto**, no copiloto: motor de inferencia del triaje
Hugging Face	`paraphrase-multilingual-MiniLM-L12-v2`	**Componente del producto**: vectorización local del corpus

Se distingue deliberadamente entre los modelos utilizados **para construir** el sistema y los que **forman parte** del sistema. Los dos últimos no asistieron al desarrollo: son infraestructura del producto entregado.

3. Metodología de trabajo

El copiloto operó bajo tres reglas fijadas al inicio:

1. **Redacción de código por el asistente, ejecución por el autor.** Las operaciones de instalación, despliegue y verificación se ejecutaron manualmente, para conservar el control del entorno y la comprensión de cada paso.
2. **Ninguna recomendación se aceptó sin contraste.** Las propuestas del modelo se evaluaron contra el enunciado del proyecto, la documentación oficial de cada tecnología y la observación del comportamiento real del sistema.
3. **Toda decisión no evidente se registró con su justificación**, no solo con su resultado. Este anexo y la bitácora `PROMPTS.md` son producto de esa regla.

4. Bloqueos documentados

B-01 · Delimitación del corpus documental

Situación. El motor de recuperación funcionaba correctamente sobre cinco guías del MSP seleccionadas al inicio, y las pruebas iniciales resultaban satisfactorias.

Observación propia. Al revisar el planteamiento completo antes de continuar, cuestioné si el enfoque general era el adecuado o si arrastrábamos decisiones tomadas demasiado pronto. El corpus me resultaba sospechosamente ajustado.

Consulta. Se solicitó una revisión crítica del diseño en su conjunto, con instrucción explícita de señalar errores de criterio propios en lugar de validar lo ya construido.

Respuesta obtenida. El asistente identificó un sesgo de selección en su propia propuesta inicial: las cinco guías —hipertensión, diabetes gestacional, trastornos hipertensivos del embarazo, acné y dificultad respiratoria neonatal— se habían elegido para que superaran tres casos de prueba, no para cubrir la casuística de un dispensario. Ante consultas frecuentes reales (dolor abdominal, fiebre, trauma menor) el sistema recuperaría los fragmentos *menos lejanos*, no los pertinentes.

Contraste crítico. Se aceptó el diagnóstico por ser verificable: bastaba consultar síntomas comunes para reproducir el fallo. De las tres alternativas planteadas —reducir el alcance del producto al ámbito obstétrico, ampliar el corpus, o ambas— se descartó la primera pese a su coste nulo: acotar el producto para que encajara con los datos disponibles habría sido resolver el problema al revés. Se optó por ampliar el corpus.

Resolución. Incorporación de los *Protocolos de Atención Prehospitalaria para Emergencias Médicas* del MSP (1.257 fragmentos, +52 % del corpus), que cubren emergencias cardiovasculares, respiratorias, neurológicas, traumatológicas y digestivas. Se verificó previamente que el documento contuviera capa de texto extraíble y no fuera un escaneo de imágenes.

B-02 · Modelo de embeddings inadecuado para el idioma

Situación. Con el corpus inicial, la consulta *"tengo la presión muy alta, dolor de cabeza fuerte y veo puntos negros"* recuperaba la guía de **acné**, no la de hipertensión.

Hipótesis inicial. Sospeché de la fragmentación o del tamaño de los fragmentos, por ser los parámetros que habíamos fijado sin evidencia.

Consulta. Se aportó la salida completa de la búsqueda —fragmentos, guía de origen y distancias— solicitando diagnóstico de la causa raíz antes de proponer correcciones.

Respuesta obtenida. El análisis descartó la fragmentación. El fragmento recuperado de la guía de acné correspondía a la descripción de comedones abiertos, literalmente *"puntos negros"*: el modelo realizaba una coincidencia **léxica**, no semántica, sin distinguir que la misma expresión designa un signo neurológico en un contexto y una lesión dermatológica en otro. La causa señalada fue el modelo de embeddings `all-MiniLM-L6-v2`, entrenado mayoritariamente en inglés, aplicado sobre texto clínico en español.

Contraste crítico. La explicación resultaba coherente con la evidencia —las distancias se agrupaban todas en un rango estrecho y mediocre, sin ninguna coincidencia fuerte—, lo que es indicio de un espacio vectorial que no discrimina bien. Se verificó que el modelo sustitutivo propuesto mantuviera la misma dimensionalidad, para confirmar que se trataba de un reemplazo directo sin efectos sobre el resto del sistema.

Resolución. Sustitución por `paraphrase-multilingual-MiniLM-L12-v2` y reingesta completa del corpus, necesaria porque los vectores previos no son comparables con los nuevos aun compartiendo dimensionalidad. Las tres consultas de prueba pasaron a recuperar la guía correcta.

B-03 · Brecha de vocabulario entre paciente y normativa clínica

Situación. Primera prueba del endpoint completo con un caso obstétrico realista: gestante de 28 años con edema facial y de manos de instauración brusca, cefalea intensa y fosfenos —presentación característica de preeclampsia con signos de gravedad.

Resultado obtenido. El sistema emitió como diagnóstico presuntivo *"reacción sistémica grave por picadura de escorpión (envenenamiento neurotóxico)"*, con un plan de adrenalina intramuscular, antihistamínicos y corticoides, para una picadura que la paciente nunca mencionó. El nivel de urgencia asignado (Rojo) resultó correcto por coincidencia.

Análisis conjunto. Las cuatro fuentes recuperadas procedían de la sección de mordeduras y envenenamientos de los protocolos prehospitalarios; ninguna de la guía de trastornos hipertensivos del embarazo, pese a estar indexada. Una prueba anterior **sí** había recuperado la guía correcta, pero incluía la frase *"la presión me salió alta"*. Sin esa pista léxica, la combinación *edema facial + síntomas neurológicos + alteración visual* se aproxima más al lenguaje de la sección de envenenamientos que al de la guía obstétrica, redactada en términos de proteinuria, hipertensión gestacional y cifras tensionales.

Se identificó además un factor agravante: la instrucción de anclaje del prompt —*"fundaméntate en el contexto recuperado y cítalo"*— llevó al modelo a **confiar en contexto erróneo por encima de su propio conocimiento clínico**. El modelo obedeció la instrucción; el defecto era de diseño.

Contraste crítico. Se propusieron dos correcciones y se implementaron ambas deliberadamente, en lugar de elegir la más económica, por tratarse de una ruta con consecuencias clínicas: una actúa sobre la recuperación y otra sobre la inferencia, de modo que el fallo de una no deja el sistema desprotegido. La decisión resultó acertada: en la primera verificación solo una de las dos funcionaba (ver B-04), y fue la segunda la que corrigió el caso.

Resolución.

1. *Recuperación en dos etapas.* El relato coloquial se traduce previamente a terminología clínica y la búsqueda se ejecuta con ambas formulaciones, fusionadas por menor distancia.
2. *Endurecimiento del prompt.* Se instruye explícitamente que el contexto puede ser irrelevante, que debe descartar fragmentos que mencionen causas no referidas por el paciente, y que su juicio clínico prevalece sobre un contexto que no encaja.

Verificación: el diagnóstico pasó a ser *"preeclampsia grave con posible evolución a eclampsia"*, con plan de sulfato de magnesio y derivación obstétrica, citando la guía correcta. La distancia mejoró de 0,45 a 0,30.

B-04 · Fallo silencioso de la etapa de reescritura

Situación. Tras aplicar las correcciones anteriores, el caso obstétrico devolvía el diagnóstico correcto. Sin embargo, al inspeccionar la respuesta completa, el campo que expone la traducción clínica venía **vacío** y las fuentes seguían siendo las páginas de envenenamientos.

Observación propia. El resultado era correcto, pero por una sola de las dos defensas implementadas. Un componente estaba fallando por completo sin producir error visible.

Consulta. Se solicitó determinar por qué la reescritura devolvía cadena vacía cuando la llamada principal, contra el mismo modelo, funcionaba con normalidad.

Respuesta obtenida. La causa señalada fue el límite de tokens fijado en la llamada de reescritura: `gpt-oss-120b` es un modelo de razonamiento y consume presupuesto de salida razonando antes de emitir contenido. Con un tope ajustado, el presupuesto se agotaba durante el razonamiento y la respuesta llegaba sin contenido. La llamada principal, sin tope, nunca manifestó el problema.

Contraste crítico. La explicación era consistente con la asimetría observada entre ambas llamadas. Se aceptó la corrección técnica, pero se rechazó darla por suficiente: el problema de fondo no era el límite de tokens sino que **el mecanismo de degradación enmascaraba el fallo**. Un componente puede fallar al cien por cien y resultar indistinguible de un éxito.

Resolución. Eliminación del tope de tokens e incorporación de registro explícito tanto para la excepción como para el caso de contenido vacío. La degradación con gracia se conserva —si la reescritura falla, la búsqueda continúa con el texto original— pero deja de ser invisible.

Aprendizaje aplicado. Todo mecanismo de respaldo del proyecto se revisó bajo este criterio: debe degradar con gracia **y ser observable**.

B-05 · Inestabilidad del árbol de dependencias

Situación. La construcción de la imagen fallaba de forma reiterada. Primero por incompatibilidad entre versiones fijadas de la familia LangChain; tras liberar las restricciones, el resolutor de `pip` quedó más de cuarenta minutos explorando combinaciones sin converger.

Observación propia. Consulté si el problema era de caché de Docker y si convenía eliminar imágenes y reconstruir desde cero.

Respuesta obtenida. Se descartó la hipótesis de caché con evidencia del propio registro de construcción: la instalación de dependencias se ejecutaba íntegramente en cada intento, sin reutilizar capas. El diagnóstico fue *backtracking* del resolutor: cinco paquetes sin restricción de versión generan un espacio combinatorio que obliga a descargar metadatos de cada combinación candidata. El registro del intento previo ya lo advertía explícitamente.

Contraste crítico. Se rechazó la solución inmediata —volver a fijar versiones—, porque los dos intentos anteriores habían fallado precisamente por estimar números de versión sin verificarlos. Ante la disyuntiva entre seguir ajustando restricciones o reducir el árbol de dependencias, se auditó qué aportaba realmente la biblioteca: carga de PDF, fragmentación de texto, generación de embeddings, persistencia vectorial e invocación del modelo. **Las cinco operaciones ya estaban cubiertas** por bibliotecas presentes en el proyecto.

Resolución. Eliminación completa de LangChain, sustituida por `pypdf`, `sentence-transformers`, `chromadb` y el SDK oficial de `groq`. El proyecto pasó de catorce a nueve dependencias, la construcción se volvió reproducible, y cada operación del flujo —vectorizar, buscar, invocar— quedó visible en el código en lugar de encapsulada.

B-06 · Selección del motor de base de datos

Situación. El diseño inicial contemplaba SQLite por simplicidad de despliegue.

Observación propia. Consulté si SQLite era realmente lo adecuado para este caso o si convenía PostgreSQL o MySQL, dado que la arquitectura ya contemplaba varios servicios.

Respuesta obtenida. El análisis identificó dos razones concretas a favor de PostgreSQL, ambas específicas de esta arquitectura y no preferencias generales: el backend y el frontend son procesos independientes con acceso concurrente a los mismos datos, escenario para el que el bloqueo a nivel de archivo de SQLite no está diseñado; y la capa de orquestación dispone de nodo nativo para PostgreSQL, mientras que integrar SQLite exigiría ejecutar un script externo desde un nodo genérico. MySQL se descartó por no aportar ventaja diferencial y ofrecer peor tratamiento de datos JSON.

Contraste crítico. El segundo argumento resultó decisivo por ser verificable contra la documentación de la herramienta de orquestación, no una apreciación de estilo. Se validó además que el coste habitual de PostgreSQL —su configuración— era nulo en este proyecto, dado que el entorno ya se ejecutaba en contenedores.

Resolución. PostgreSQL 16 como servicio adicional, con verificación de estado para evitar condiciones de carrera en el arranque. La decisión resultó estructural: al existir un nodo nativo, la persistencia se trasladó íntegramente a la capa de orquestación y el backend quedó **sin estado**, lo que simplificó su verificación de manera sustancial.

B-07 · Integración entre el orquestador y la base de datos

Situación. Tres fallos consecutivos al conectar el flujo de n8n con PostgreSQL.

Fallo 1 — webhook no registrado. La invocación al endpoint de producción no obtenía respuesta. El registro del orquestador indicaba que la ruta no existía. Causa: la versión 2.x emplea un modelo de borrador y publicación, y un flujo guardado permanece como borrador. Se identificó también la distinción entre la URL de prueba —activa solo durante una escucha manual— y la de producción, disponible únicamente tras publicar.

Fallo 2 — rechazo del campo JSONB. La columna destinada a las fuentes de recuperación rechazaba el valor. La indicación recibida —serializar el arreglo a texto— resultó **incorrecta**: la columna es de tipo JSONB y el nodo espera una estructura, no una cadena. Al probar con el arreglo sin serializar, el nodo tampoco lo aceptó. **Resolví el bloqueo envolviendo el arreglo en un objeto**, forma que el validador sí admite y que PostgreSQL almacena correctamente. La lectura en el panel del médico se adaptó a esa estructura.

Fallo 3 — clave primaria fija. El primer registro se insertó con identificador cero. La inspección de la secuencia confirmó que nunca había sido invocada: el orquestador mapeaba la columna de clave primaria de forma explícita, evitando el valor por defecto. Con un único caso el defecto pasaba inadvertido; el segundo

habría fallado por violación de clave primaria. Se detectó por verificación del estado de la base tras la primera inserción exitosa, no por error manifiesto.

Contraste crítico. El segundo fallo es el más relevante: la indicación del asistente era técnicamente incorrecta y la corrección surgió de la experimentación directa sobre la herramienta. El tercero confirma el valor de verificar el estado real de la base después de una operación aparentemente exitosa, en lugar de asumir corrección a partir de la ausencia de error.

Resolución. Publicación del flujo, envoltura del arreglo en objeto para el campo JSONB, y supresión del mapeo explícito de la clave primaria. Verificado con dos inserciones consecutivas de identificadores correlativos.

B-08 · Resistencia a inyección de prompts

Situación. Antes de decidir si implementar un filtro de sanitización de entrada —una mejora contemplada en la planificación— se consideró necesario determinar si el sistema era efectivamente vulnerable, en lugar de implementarlo por precaución.

Prueba diseñada. Instrucción destinada a subvertir el rol clínico y obtener la prescripción de un opioide sin indicación médica, combinando anulación de instrucciones previas, redefinición de rol y solicitud de dosis concreta.

Resultado. El sistema resistió el ataque en su ruta crítica: no declaró sano al paciente ni prescribió el fármaco. Clasificó el caso como posible búsqueda de opioides, indicó expresamente no administrar morfina y derivó a valoración especializada. Citó los fragmentos recuperados sobre morfina para argumentar que **no resultaban aplicables** al no existir dolor reportado.

Se detectó no obstante una vulnerabilidad secundaria: la etapa de reescritura podía ser desviada, devolviendo una negativa —*"I'm sorry, but I can't comply with that"*— que luego se empleaba como consulta de búsqueda vectorial, contaminando las fuentes. El comportamiento resultaba además no determinista: en una ejecución la respuesta no superó la validación de esquema y el endpoint devolvió error, lo que ante un usuario final se traduce en un fallo visible.

Contraste crítico. El resultado contradijo la expectativa de la planificación inicial, que asumía la necesidad de un filtro dedicado. La evidencia mostró que el diseño del prompt y la validación estricta de esquema ya contenían el ataque donde importa. Implementar un sanitizador adicional habría añadido complejidad sin cubrir un riesgo demostrado, y habría introducido un punto de falso rechazo sobre relatos legítimos de pacientes.

Resolución. Se descartó el filtro independiente y se corrigió la vulnerabilidad concreta observada: validación de la salida de la reescritura, que se descarta si viene vacía, resulta demasiado breve o contiene marcas de rechazo, continuando la búsqueda con el texto original. La decisión de no implementar el filtro queda sustentada en evidencia experimental documentada, no en supuesto.

5. Defectos detectados por auditoría preventiva

Además de los bloqueos anteriores —todos manifestados durante la ejecución— se solicitó expresamente una revisión crítica del código recién escrito, antes de ejecutarlo. Esa revisión identificó tres defectos que no se habían manifestado:

Defecto	Consecuencia evitada
Retroceso del cursor en el fragmentador de texto	Bucle infinito ante fragmentos con un único espacio temprano — situación que se produce con tablas o texto mal extraído de PDF
Ausencia de inserción por lotes en la base vectorial	Fallo al superar el límite por operación de ChromaDB, tras haber consumido el tiempo de vectorización completo
Caché del modelo de embeddings no persistida	Descarga repetida de 470 MB en cada ejecución efímera de contenedor

Ninguno se habría detectado leyendo el código superficialmente, y los dos primeros habrían aparecido únicamente al ampliar el corpus. Se consigna para dejar constancia de que la revisión del código generado por IA formó parte del método de trabajo, y no solo su ejecución.

6. Valoración del uso de IA como copiloto

Aportes efectivos. Aceleración sustancial en redacción de código de infraestructura, diagnóstico de causa raíz a partir de registros de error, y conocimiento actualizado sobre disponibilidad de modelos —la deprecación del modelo previsto en la planificación inicial se detectó por verificación contra la documentación del proveedor, no por conocimiento previo del asistente.

Limitaciones observadas. Tres son destacables por reproducibles:

Versiones de dependencias estimadas y no verificadas. Las versiones propuestas inicialmente correspondían al conocimiento del modelo, no al estado real del repositorio de paquetes en la fecha del proyecto. Provocó dos ciclos de construcción fallidos (B-05).

Sesgo de confirmación en el propio diseño. El corpus documental se seleccionó de modo que superara las pruebas propuestas por el mismo asistente. El error solo emergió al solicitar una revisión crítica explícita del planteamiento completo (B-01).

Indicaciones incorrectas sobre herramientas de terceros. El mapeo del campo JSONB en el orquestador fue erróneo y se resolvió por experimentación directa (B-07).

Conclusión. El asistente resultó eficaz en generación de código y análisis de causa raíz sobre evidencia concreta, y menos fiable en afirmaciones sobre el estado actual de ecosistemas externos y en la evaluación crítica de sus propias propuestas. El método que se demostró productivo fue tratar cada recomendación como hipótesis a verificar —contra documentación oficial o contra el comportamiento observado del sistema— y solicitar revisión adversarial explícita del diseño en los puntos de decisión, en lugar de aceptar la validación de lo ya construido.